FITNESS TRACKER: INTERFACES GRÁFICAS DE USUARIO

Rubén Hernández Molina / Rubenhernandezm@usal.es

Índice

1. Manual De Usuario:.....3
2. Manual del Programador.....9
3. Bibliografía.....13

1. Manual De Usuario:

Este manual de usuario está diseñado para proporcionar una guía completa sobre el uso de la aplicación. La herramienta ha sido diseñada para gestionar y analizar de manera eficiente los ejercicios físicos, permitiendo a los usuarios registrar, visualizar y monitorear sus entrenamientos de forma intuitiva.

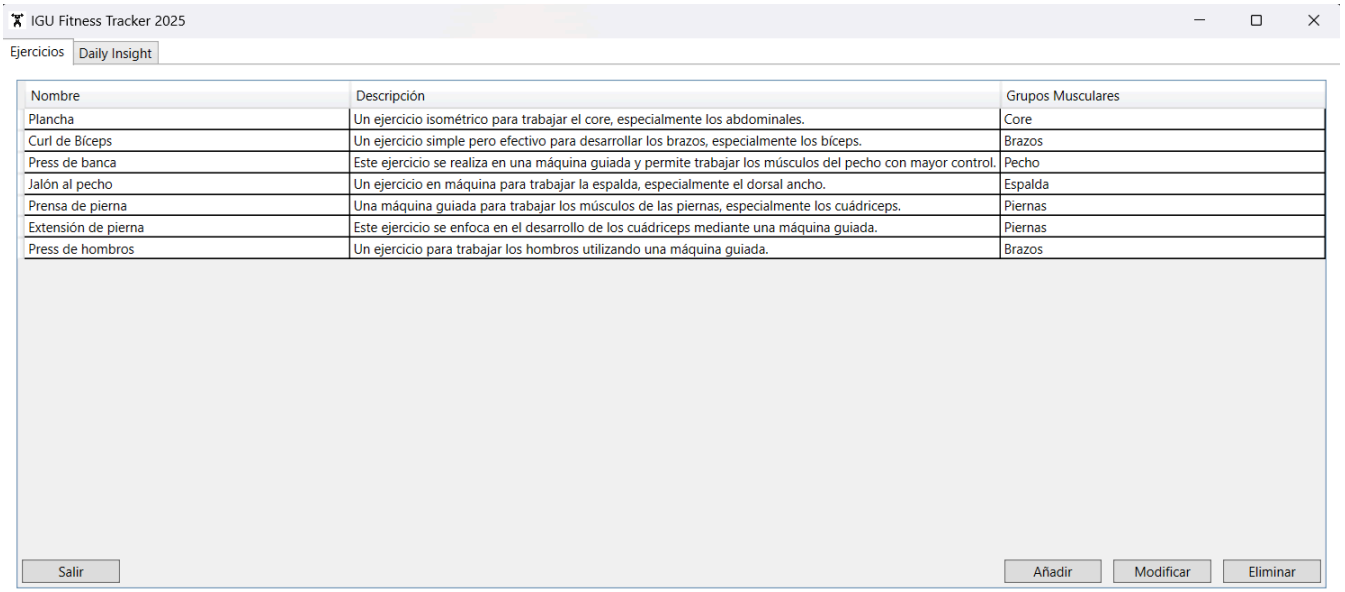
En las siguientes páginas se detallarán las principales funcionalidades de la aplicación.

El objetivo de este manual es garantizar que cualquier usuario pueda utilizar la aplicación de manera eficaz, maximizando su potencial para el seguimiento del rendimiento físico.

Al ejecutar la aplicación lo primero que encontramos es la ventana principal del programa, la cual nos muestra dos pestañas(Figura 1):

1. Ejercicios: En esta pestaña, podemos ver una lista completa de los ejercicios registrados, junto con una breve descripción y los grupos musculares involucrados. Desde aquí es posible también añadir nuevos ejercicios, modificar los existentes o eliminarlos.
2. Daily Insight(Figura 2): Esta pestaña muestra un gráfico radial interactivo que resume las repeticiones(solo muestra hasta un máximo de 100) acumuladas para cada grupo muscular en una fecha específica, inicialmente parte de la fecha actual.También cuenta con tres botones de navegación que nos permiten cambiar fácilmente entre días.Este gráfico se actualiza dinámicamente cuando seleccionamos una ejecución concreta de un ejercicio, pasando a referir la fecha de dicha ejecución.

La ventana principal está diseñada para ofrecer una experiencia intuitiva y organizada, permitiendo un acceso rápido a todas las funcionalidades claves de la aplicación.



Nombre	Descripción	Grupos Musculares
Plancha	Un ejercicio isométrico para trabajar el core, especialmente los abdominales.	Core
Curl de Bíceps	Un ejercicio simple pero efectivo para desarrollar los brazos, especialmente los bíceps.	Brazos
Press de banca	Este ejercicio se realiza en una máquina guiada y permite trabajar los músculos del pecho con mayor control.	Pecho
Jalón al pecho	Un ejercicio en máquina para trabajar la espalda, especialmente el dorsal ancho.	Espalda
Prensa de pierna	Una máquina guiada para trabajar los músculos de las piernas, especialmente los cuádriceps.	Piernas
Extensión de pierna	Este ejercicio se enfoca en el desarrollo de los cuádriceps mediante una máquina guiada.	Piernas
Press de hombros	Un ejercicio para trabajar los hombros utilizando una máquina guiada.	Brazos

Figura 1

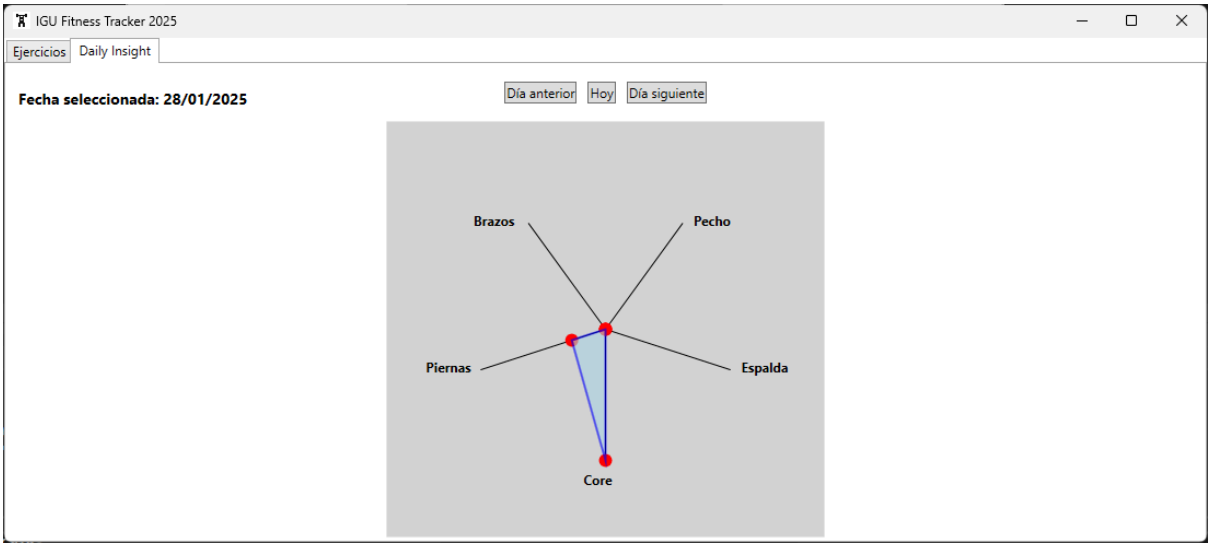


Figura 2

Como hemos indicado anteriormente, la pestaña de ejercicios permite añadir,modificar y eliminar ejercicios, pulsando sus respectivos botones. Para los casos de añadir(Figura 3) y modificar(Figura 4), se abre una ventana auxiliar(modal) la cual maneja las excepciones de que en caso de que no se rellene ningún campo(Figura 5) o que alguno de los campos introducidos no sean válidos(Figura 6) muestran mensajes de advertencia.

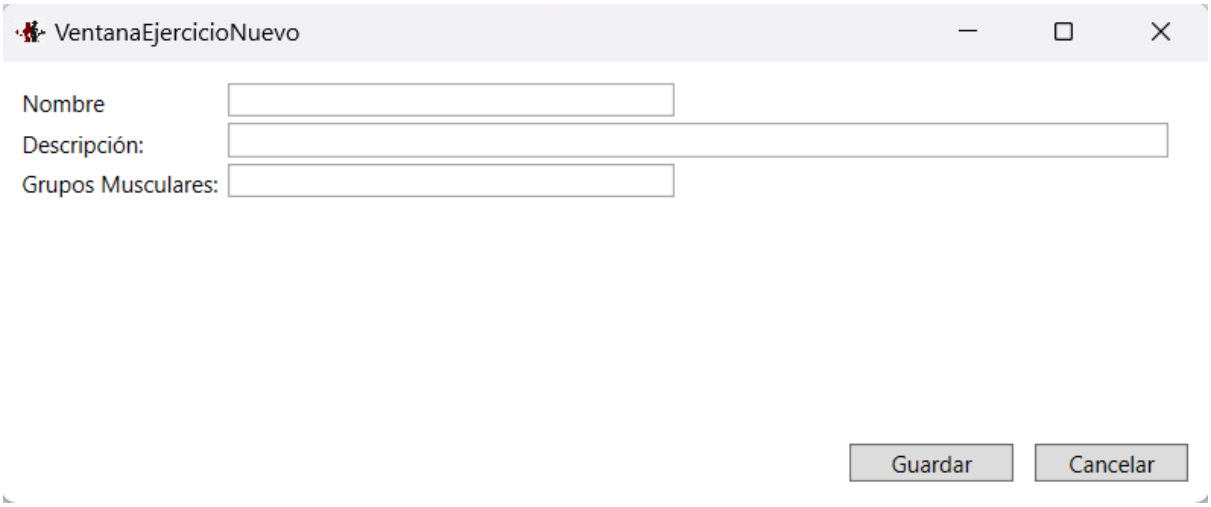


Figura 3

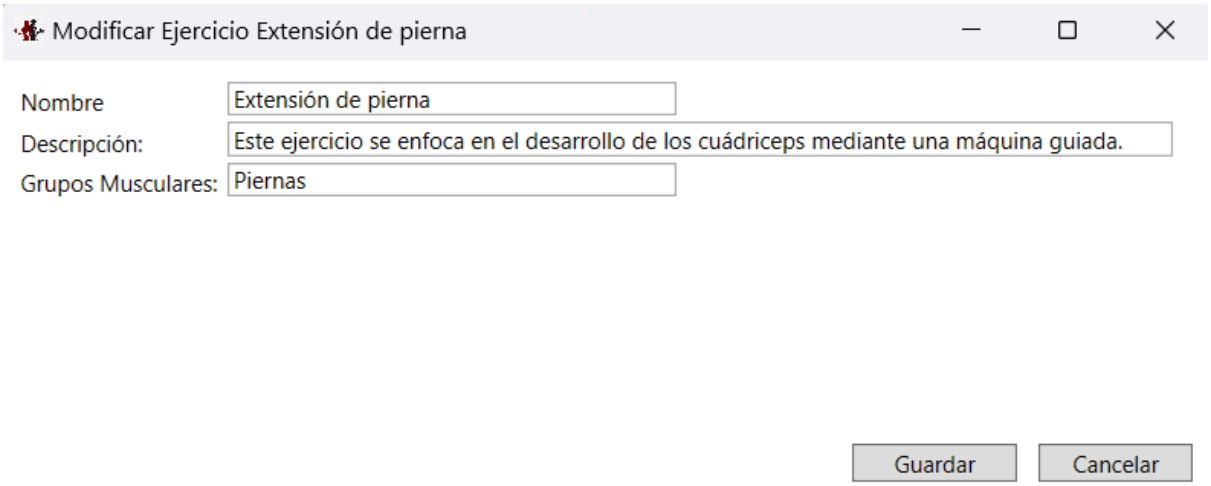


Figura 4

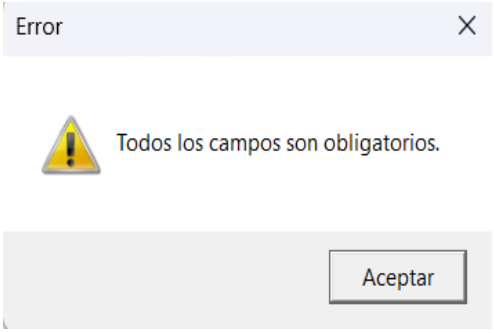


Figura 5

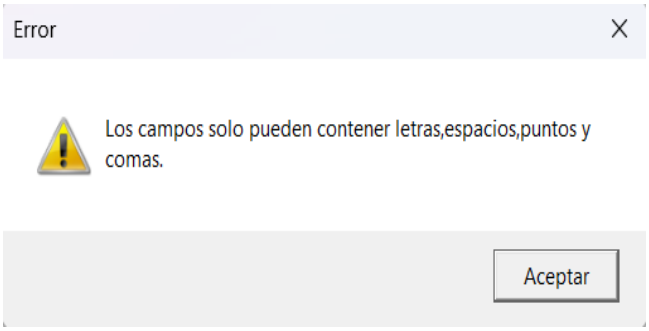


Figura 6

En el caso de eliminar simplemente muestra un mensaje de confirmación de eliminación(Figura 7).

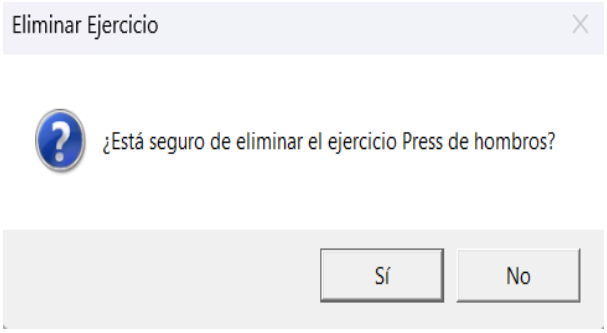


Figura 7

Si realizamos doble click sobre cualquiera de los ejercicios de la pestaña de Ejecuciones, se abrirá una ventana(no modal) con los detalles del ejercicio(Figura 8):repeticiones, peso(kg) y fecha y hora.

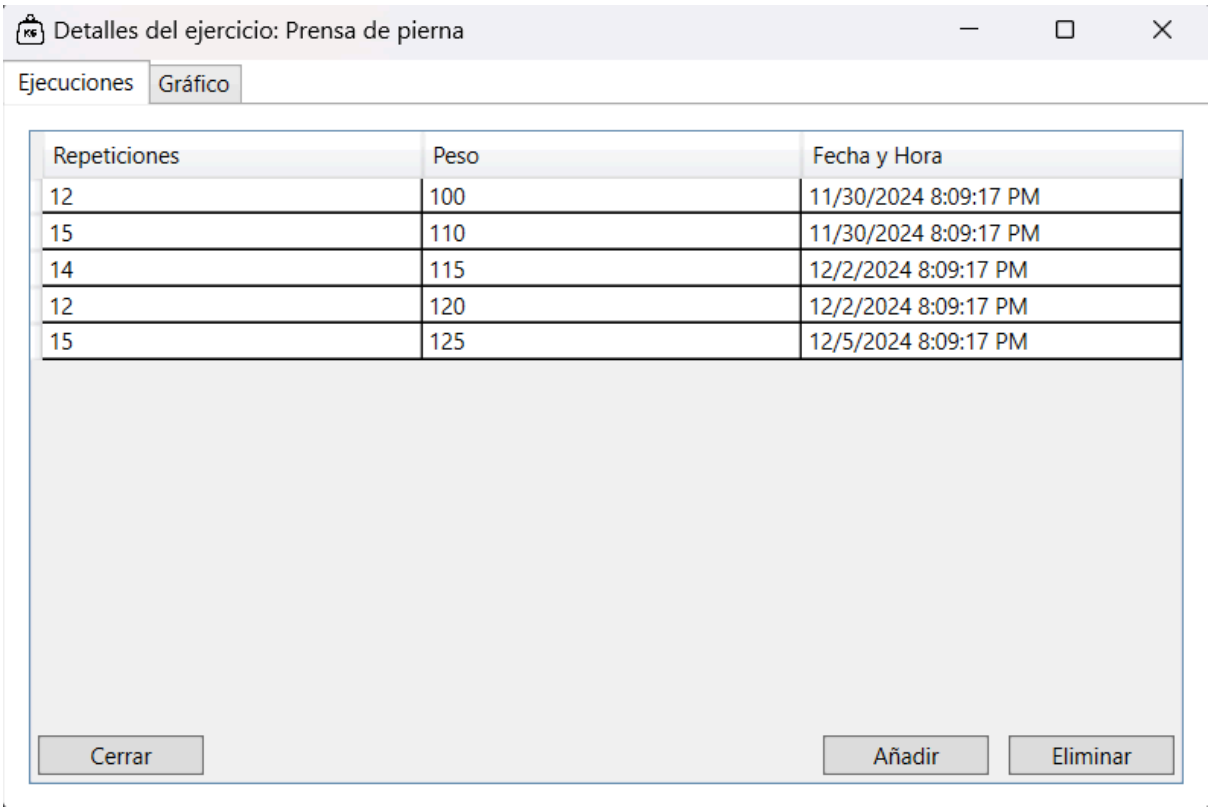


Figura 8

Esta ventana muestra una lista de las ejecuciones realizadas para el ejercicio seleccionado, permitiéndonos también añadir y eliminar

ejecuciones. Al ser una ventana de carácter no modal si seleccionamos otro ejercicio en la ventana principal, pasará a referir las ejecuciones de este ejercicio de manera automática.

A la hora de añadir ejecuciones, se abrirá una ventana(modal) para introducir los campos de la nueva ejecución(Figura 9), la cual también maneja las excepciones de que el formato sea erróneo(Figura 10,Figura 11 y Figura 12).

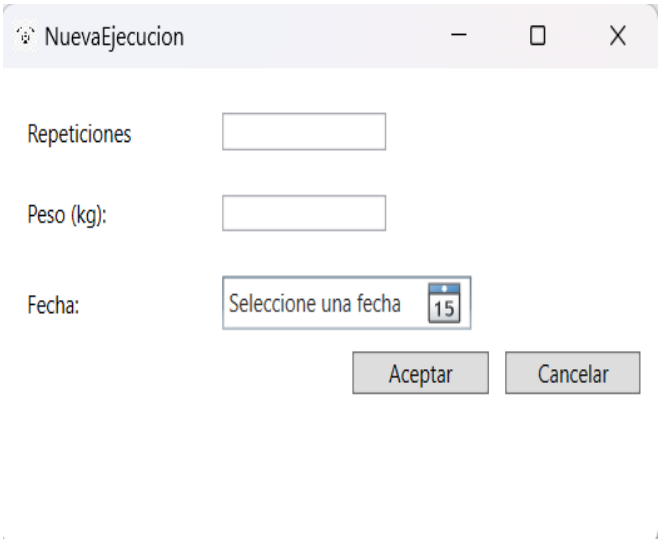


Figura 9

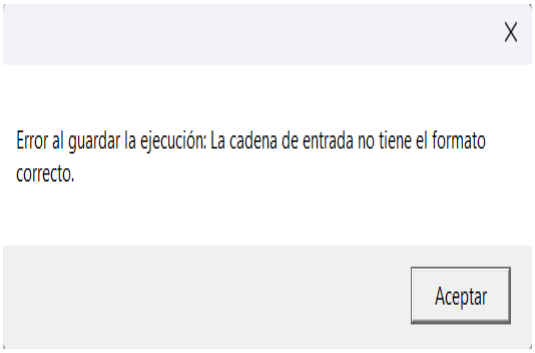


Figura 10

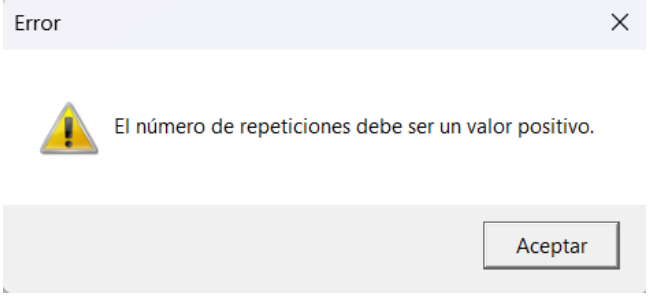


Figura 11

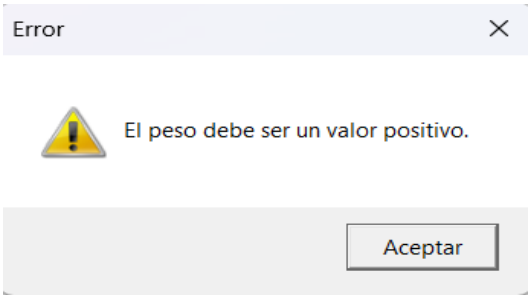


Figura 12

En el caso de eliminar, como anteriormente, mostrará un mensaje de confirmación de eliminación de la ejecución seleccionada(Figura 13).

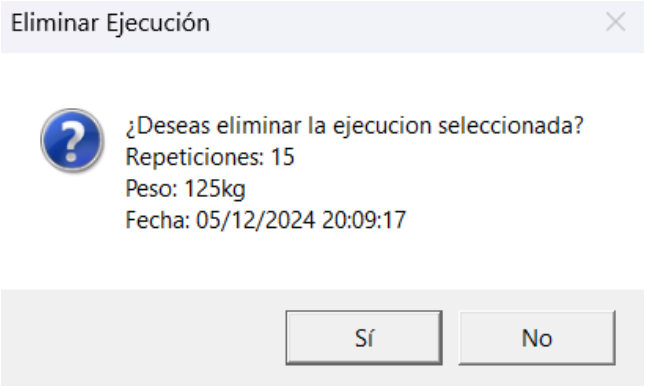


Figura 13

Si clicamos en la pestaña de Gráfico de la figura 8, pasará a mostrar un gráfico de barras con las repeticiones y peso de cada ejecución agrupadas por fecha(Figura 14). Cuenta con un ToolTip que permite ver de forma más clara las repeticiones y el peso.

Cabe destacar que tanto la adición como la eliminación de ejecuciones afectan tanto al gráfico de ejecuciones como al de ejercicios, actualizándose ambos de manera dinámica.

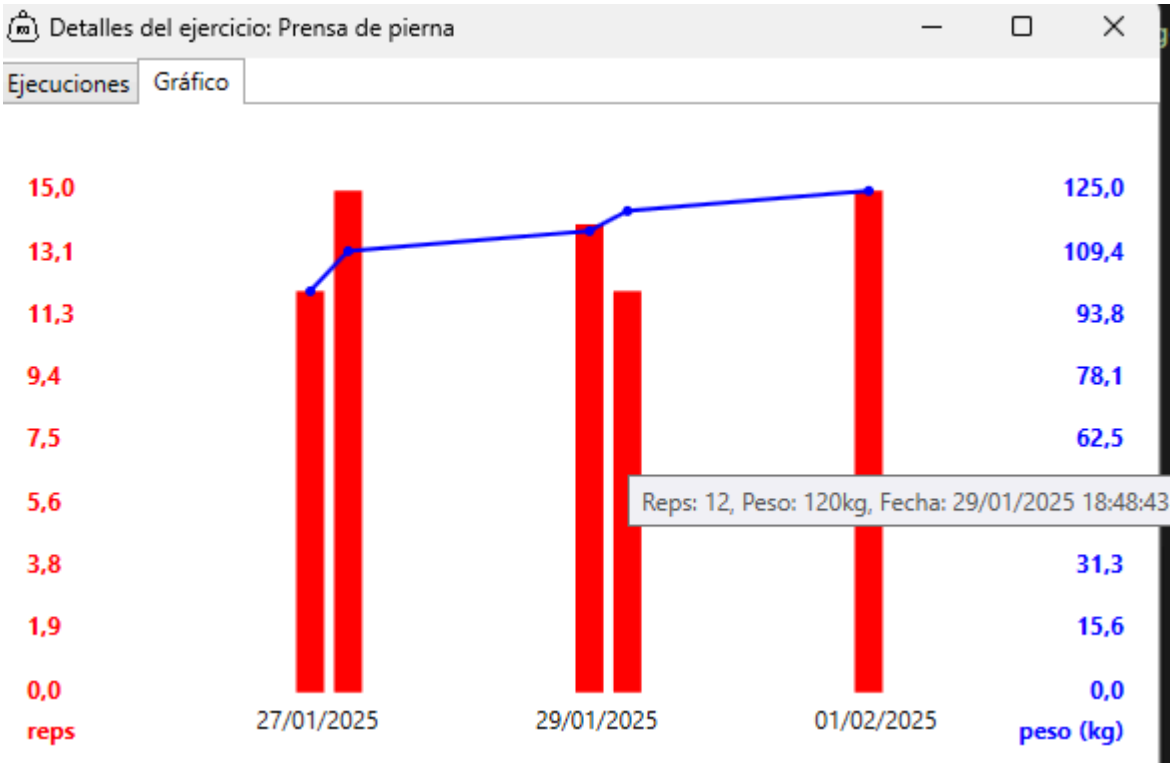


Figura 14

2. Manual del Programador

Este manual tiene como objetivo proporcionar una descripción detallada de la arquitectura, diseño y funcionamiento interno de la aplicación.
Se incluyen explicaciones sobre la estructura del código, los principales componentes de la solución(ventanas,clases y modelos de datos) y las interacciones entre ellos.

Además se detallan las herramientas y tecnologías utilizadas, como WPF para la interfaz gráfica y C# para la lógica de la aplicación.

Se adjunta a continuación para comenzar a detallar la aplicación, el conjunto de los elementos que componen el sistema(Figura 1):

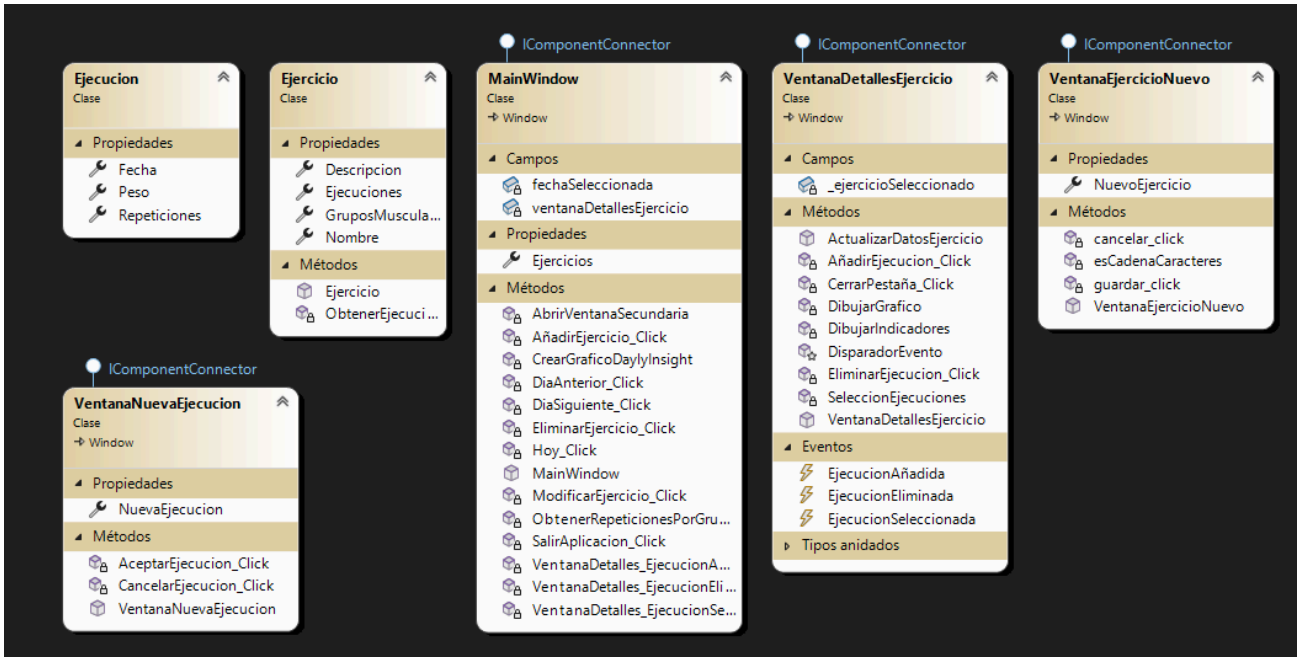


Figura 1

<u>Clases:</u>
Para la realización de la aplicación se han definido dos clases: Ejercicio y Ejecución.
La clase "<u>Ejecución</u>" contiene los campos que se necesitan guardar para cada ejecución de un ejercicio, es decir el número de repeticiones, el peso en kilogramos y la fecha de ejecución, la cual definimos con el tipo

de dato “DateTime” para almacenar la fecha y hora correspondientes.(Figura 2)

```
public class Ejecucion
{
    18 referencias
    public int Repeticiones { get; set; }
    16 referencias
    public double Peso { get; set; }
    17 referencias
    public DateTime Fecha { get; set; }

}
```

Figura 2

La clase “Ejercicio” contiene los campos que comprenden a los ejercicios, los cuales son, el nombre, una breve descripción y los grupos musculares a los que afecta. Añadimos también en esta clase una colección de ejecuciones para añadir las predeterminadas con las que debe contar la aplicación al arrancarse mediante el uso de “ObservableCollection<>”, lo cual permite la actualización automática de la tabla enlazada.(Figura 3)

```
public class Ejercicio
{
    8 referencias
    public string Nombre { get; set; }
    4 referencias
    public string Descripcion { get; set; }
    5 referencias
    public string GruposMusculares { get; set; }
    7 referencias
    public ObservableCollection<Ejecucion> Ejecuciones { get; set; }
```

Figura 3

Esta clase cuenta también con una función en la que se añaden a la colección de ejecuciones las predeterminadas.

Ventanas:

La aplicación cuenta con una ventana principal en la cual se muestra la información principal contenida de cada ejercicio y el gráfico radial para cada fecha, una ventana secundaria de carácter no modal para visualizar todo lo relacionado con las ejecuciones y sus respectivos

gráficos, y dos ventanas de carácter modal para la adición y modificación de ejecuciones y ejercicios.

La ventana secundaria se abre al realizar un doble click en cualquiera de los ejercicios de la ventana principal(Figura 4).

```
MouseDoubleClick="AbrirVentanaSecundaria"
```

Figura 4

La función para abrir esta ventana no modal en detalle es la que se muestra a continuación(Figura 5):

```
1 referencia
private void AbrirVentanaSecundaria(object sender, MouseButtonEventArgs e)
{
    try
    {
        if (EjerciciosDataGrid.SelectedItem is Ejercicio ejercicioSeleccionado)
        {
            // Verifica si la ventana secundaria está abierta
            if (ventanaDetallesEjercicio == null || !ventanaDetallesEjercicio.IsVisible)
            {
                // Crear nueva ventana secundaria
                ventanaDetallesEjercicio = new VentanaDetallesEjercicio(ejercicioSeleccionado)
                {
                    Owner = this // Establecer la ventana principal como propietaria para que se cierre al cerrar la principal
                };
                ventanaDetallesEjercicio.Show();
                ventanaDetallesEjercicio.Closed += (s, args) => ventanaDetallesEjercicio = null;

                //suscribimos al evento de la ventana secundaria para que se actualice el grafico radial
                ventanaDetallesEjercicio.EjecucionSeleccionada += VentanaDetalles_EjecucionSeleccionada;
                // Suscribir al evento de adición de ejecución
                ventanaDetallesEjercicio.EjecucionAñadida += VentanaDetalles_EjecucionAñadida;
                // Suscribir al evento de eliminación de ejecución
                ventanaDetallesEjercicio.EjecucionEliminada += VentanaDetalles_EjecucionEliminada;
            }
            else
            {
                // Reutilizar la ventana secundaria abierta y actualizar los datos
                ventanaDetallesEjercicio.ActualizarDatosEjercicio(ejercicioSeleccionado);
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ocurrió un error: {ex.Message}", "Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Figura 5

Como podemos observar en la captura anterior, en esta función nos suscribimos a los tres eventos con los que cuenta el sistema, los cuales actualizan el gráfico radial de forma automática si se añaden o eliminan ejecuciones o pasa a referenciar el gráfico para la fecha de una ejecución concreta seleccionada.

Las otras dos ventanas de la aplicación, como ya indiqué anteriormente, son de carácter modal y ambas se abren de la misma manera, los respectivos botones tienen su funcionalidad.

A través de la ventana principal, clicando en el botón de añadir(Figura 6) desplegamos la ventana “VentanaEjercicioNuevo” , la cual usamos para registrar un ejercicio nuevo, cuenta con ciertas comprobaciones para evitar que la entrada de datos sea errónea y cuyo código en detalle es el que se muestra a continuación(Figura 7):

```
Click="AñadirEjercicio_Click"
```

Figura 6

```
public partial class VentanaEjercicioNuevo : Window
{
    2 referencias
    public Ejercicio NuevoEjercicio { get; set; }
    1 referencia
    public VentanaEjercicioNuevo()
    {
        InitializeComponent();
    }

    2 referencias
    private void guardar_click(object sender,RoutedEventArgs e)
    {
        if (string.IsNullOrWhiteSpace(txtNombre.Text) || string.IsNullOrWhiteSpace(txtDescripcion.Text)
            ||string.IsNullOrWhiteSpace(txtGruposMusculares.Text))
        {
            MessageBox.Show("Todos los campos son obligatorios.", "Error", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        if (!esCadenaCaracteres(txtNombre.Text)||!esCadenaCaracteres(txtDescripcion.Text)
            ||!esCadenaCaracteres(txtGruposMusculares.Text))
        {
            MessageBox.Show("Los campos solo pueden contener letras,espacios,puntos y comas.", "Error", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        NuevoEjercicio = new Ejercicio(

            txtNombre.Text,
            txtDescripcion.Text,
            txtGruposMusculares.Text
        );

        DialogResult = true;
        Close();

    }

    0 referencias
    private void cancelar_click(object sender, RoutedEventArgs e)
    {
        DialogResult = false;
        Close();
    }

    3 referencias
    private bool esCadenaCaracteres(string cadena)
    {
        return  cadena.All(c=>char.IsLetter(c)||char.IsWhiteSpace(c)||c=='.'||c==',');
    }
}
```

Figura 7

De la misma manera, clicando en el botón de añadir en la ventana secundaria, se desplegará la ventana “VentanaNuevaEjecucion” la cual nos permite introducir los datos de una nueva ejecución para un ejercicio.Se muestra el fragmento de código(Figura 8):

```
private void AñadirEjecucion_Click(object sender, RoutedEventArgs e)
{
    var nuevaEjecucionWindow = new VentanaNuevaEjecucion();
    if (nuevaEjecucionWindow.ShowDialog() == true) // Hacemos una ventana modal para que se espere a que se cierre
    {
        // Añadir la nueva ejecución al ejercicio mostrado
        _ejercicioSeleccionado.Ejecuciones.Add(nuevaEjecucionWindow.NuevaEjecucion);

        // Redibujar gráfico y actualizar tabla
        DibujarGrafico();
    }
}
```

Figura 8

Métodos más significativos:

Respecto a los métodos más significativos podemos destacar lo relacionado el evento de EjecucionSeleccionada, el cual como he comentado anteriormente por encima, lo que hace es que al seleccionar una ejecución concreta de un ejercicio, el gráfico radial de la ventana principal pasa a mostrarse para la fecha de dicha ejecución concreta. En la siguiente captura(Figura 9) muestro lo relacionado al controlador del evento, lo cual se encuentra en la ventana principal y al cual estamos suscritos en el método que mostré en la figura 5 .

```
//Declaramos el controlador del evento de la ventana secundaria
1 referencia
private void VentanaDetalles_EjecucionSeleccionada(object sender,VentanaDetallesEjercicio.EjecucionSeleccionadaEventArgs e)
{
    //actualizamos el grafico radial
    CrearGraficoDaylyInsight(e.Fecha);
}
```

Figura 9

En la ventana MostrarDetallesEjercicio encontramos todo lo demás relacionado con este evento, su manejador, disparador, delegate…(Figura 10 y Figura 11).

```
1 referencia
private void SeleccionEjecuciones(object sender,SelectionChangedEventArgs e)
{
    if(EjecucionesDataGrid.SelectedItem is Ejecucion ejecucionSeleccionada)
    {
        var args = new EjecucionSeleccionadaEventArgs(ejecucionSeleccionada.Fecha);
        //disparar el evento
        DisparadorEvento(args);
    }
}

1 referencia
protected virtual void DisparadorEvento(EjecucionSeleccionadaEventArgs e)
{
    if(EjecucionSeleccionada != null)
        EjecucionSeleccionada(this, e);
}

5 referencias
public class EjecucionSeleccionadaEventArgs : EventArgs
{
    2 referencias
    public DateTime Fecha { get; }

    1 referencia
    public EjecucionSeleccionadaEventArgs(DateTime fecha)
    {
        Fecha = fecha;
    }
}
```

Figura 10

```
public delegate void EjecucionSeleccionadaEventHandler(object sender, EjecucionSeleccionadaEventArgs e);
public event EjecucionSeleccionadaEventHandler EjecucionSeleccionada;
public event Action EjecucionEliminada;
public event Action EjecucionAñadida;
```

Figura 11

Para el resto de eventos se actúa de la misma manera que con el anterior.

El método ActualizarDatosEjercicio, también me parece otro método relevante, puesto que es el que permite que una vez abierta la ventana secundaria podamos cambiar su contenido haciendo doble clic en otro ejercicio de la tabla principal, pasando a hacer referencia a este mismo.(Figura 12)

```
1 referencia
public void ActualizarDatosEjercicio(Ejercicio ejercicioSeleccionado)
{
    _ejercicioSeleccionado = ejercicioSeleccionado; // Actualizar la referencia al ejercicio seleccionado
    this.Title = $"Detalles del ejercicio: {ejercicioSeleccionado.Nombre}";
    EjecucionesDataGrid.ItemsSource = null;
    EjecucionesDataGrid.ItemsSource = _ejercicioSeleccionado.Ejecuciones;

    // Redibujar gráfico con los datos actualizados
    DibujarGrafico();
}
```

Figura 12

Y por último, el método de AceptarEjecucion_Click, que hace referencia a añadir una nueva ejecución para un ejercicio, con sus respectivas comprobaciones para la introducción de de datos y cómo se crea una nueva ejecución.(Figura 13).

```
private void AceptarEjecucion_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Intentar convertir los valores ingresados
        if (!int.TryParse(Repeticionestxbx.Text, out int repeticiones) || repeticiones < 0)
        {
            MessageBox.Show("El número de repeticiones debe ser un valor positivo.", "Error", MessageBoxButton.OK, MessageBoxImage.Warning)
            Repeticionestxbx.Focus();
            return;
        }

        if (!double.TryParse(Pesotxbx.Text, out double peso) || peso < 0)
        {
            MessageBox.Show("El peso debe ser un valor positivo.", "Error", MessageBoxButton.OK, MessageBoxImage.Warning);
            Pesotxbx.Focus();
            return;
        }

        // Verificar la fecha
        DateTime fecha = FechaPicker.SelectedDate ?? DateTime.Now;

        // Crear la nueva ejecución
        NuevaEjecucion = new Ejecucion
        {
            Repeticiones = repeticiones,
            Peso = peso,
            Fecha = fecha
        };

        DialogResult = true; // Cierra la ventana con éxito
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al guardar la ejecución: " + ex.Message, "Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Figura 13

3. Bibliografía

1.WPF Tutorial. "The DatePicker Control". *WPF-Tutorial.com*. Disponible en: https://wpf-tutorial.com/es/497/controles-adicionales/the-datepicker-control/.

2.Microsoft Learn. "How to: Open a Message Box". *Microsoft Docs*. Disponible en: https://learn.microsoft.com/es-es/dotnet/desktop/wpf/windows/how-to-open-message-box?view=netdesktop-9.0.

3.Alura Cursos. "Manejando fechas y horas en C#". *Alura Cursos Blog*. Disponible en: https://www.aluracursos.com/blog/manejando-fechas-y-horas-en-csharp.

4.Net Mentor. "Diccionarios en C#". *Net Mentor*. Disponible en: https://www.netmentor.es/entrada/diccionarios-csharp.

5.Microsoft Learn. "System.Array.IndexOf Method". *Microsoft Docs*. Disponible en: https://learn.microsoft.com/es-es/dotnet/api/system.array.indexof?view=net-8.0.

6.Microsoft Learn. "System.Windows.Shapes.Polygon Class". *Microsoft Docs*. Disponible en: https://learn.microsoft.com/es-es/dotnet/api/system.windows.shapes.polygon?view=windowsdesktop-7.0.

7.Microsoft Learn. "How to: Create and Bind to an ObservableCollection". *Microsoft Docs*. Disponible en: https://learn.microsoft.com/es-es/dotnet/desktop/wpf/data/how-to-create-and-bind-to-an-observablecollection?view=netframeworkdesktop-4.8.

8.Chat GPT. Asistencia en la realización de gráficos visualmente correctos y atractivos, incluyendo ajustes en la colocación de etiquetas, alineación de barras y orientación de gráficos radiales.